

第 8 章 单周期 RISC-V CPU 的实现

8.1 RISC-V 概述

8.1.1 RISC-V 背景概述

RISC-V 指令架构是加州大学伯克利分校于 2010 年启动的 RISC ISA 设计的第五版成果。它秉承简洁、高效的设计理念，致力于为计算设备打造一种性能卓越且易于实现的指令集方案。RISC-V 采用开源模式，遵循 BSD 协议，这赋予了设计者极大的自由度，使其能够在没有任何专利限制的环境下充分发挥创造力，进行定制化创新和开发，从而推动了该架构在学术研究、工业应用以及开源社区等众多领域的蓬勃发展，为全球计算技术的进步注入了新的活力。

8.1.2 RISC-V 指令集概述

RISC-V 指令集架构采用模块化设计，分为基础指令集和扩展指令集。这种设计使得开发者可以根据具体应用需求选择合适的指令集模块，实现定制化的处理器设计。Base 指令集有 RV32I、RV64I、RV128I 几类，分别定义不同位宽的寄存器，适用于不同类型的计算机和应用场景。

本次 project 我们采用的是 RV32I 指令集。它定义了 32 个 32 位宽的通用寄存器，适用于通用计算和嵌入式系统。RV32I 指令集包含六种类型的指令，每种类型有不同的用途和格式：

- **R 型指令**：寄存器类型指令，用于在寄存器之间执行算术、逻辑和比较运算。
示例：add rd, rs1, rs2 ($rd = rs1 + rs2$)
- **I 型指令**：立即数类型指令或者加载指令。前者用于在寄存器和立即数（常数）之间执行算术、逻辑、移位和分支等操作。后者用于将存储器中的数据加载到寄存器中。
示例：addi rd, rs1, imm ($rd = rs1 + imm$)
示例：lw rd, rs1, offset (将 $rs1 + offset$ 的地址存储到 rd 中)
- **S 型指令**：存储类型指令，用于将寄存器中的数据存储在存储器中。
示例：sw rs2, offset(rs1) (将 rs2 的内容存储到 $rs1 + offset$ 指向的内存地址)
- **B 型指令**：分支类型指令，用于根据条件跳转到不同的代码段。
示例：beq rs1, rs2, offset (如果 $rs1 == rs2$ ，则跳转到 offset 指向的地址)
- **U 型指令**：无条件跳转类型指令，用于无条件跳转到指定的地址。
示例：lui rd, imm ($rd = imm \ll 12$)
- **J 型指令**：跳转类型指令，用于跳转到指定的地址。
示例：jal rd, offset (跳转到 offset 指向的地址，并将返回地址存储在 rd 中)

本次 project 我们只涉及到其中部分指令的构建：

- **Integer computational instructions**：主要是 R 型指令和部分 I 型指令
如：ADD, ADDI, SUB, AND, OR, XOR, 和位移指令
- **Load and store instructions**：主要是部分 I 型指令和 S 型指令
如：LW 和 SW
- **Control transfer instructions**：主要是 B 型指令和 J 型指令
如：BEQ, BNE, JAL, JALR, 和 AUIPC

8.1.3 RISC-V 与 ARM 架构的对比概述

本学期我们学习了 ARM 架构的相关知识，下面我们从几个大方向来比较 RISC-V 与 ARM 架构之间的差别：

- 架构篇幅

ARM：架构文档篇幅较长，通常有几千页。

RISC-V：架构文档相对简洁，大约 300 页左右。

- 模块化与可扩展性

ARM：不支持模块化和高度可扩展性，架构相对固定。

RISC-V：支持模块化和高度可扩展性，设计者可以根据特定应用需求添加自定义指令。

- 指令数量

ARM：指令数量繁多，不同分支之间可能存在不兼容的情况。

RISC-V：基础指令集约 40-50 条，扩展后的常用指令也不过百条。

- 易实现性

ARM：硬件实现复杂度较高，设计和验证过程较为复杂。

RISC-V：硬件实现简单，设计和验证过程相对容易。

- 市场地位与应用

ARM：在移动设备和嵌入式系统领域占据主导地位，广泛应用于智能手机、平板电脑、物联网设备等。苹果、三星、高通等大公司的产品都依赖 ARM 处理器。

RISC-V：虽然在市场中的应用还在逐步扩展，但其开源和灵活的特性使其在嵌入式系统、物联网、人工智能和高性能计算等领域具有很大的发展潜力。RISC-V 的生态系统正在迅速扩大，越来越多的公司和机构加入 RISC-V International，推出了多款 RISC-V 处理器和片上系统（SoC）。

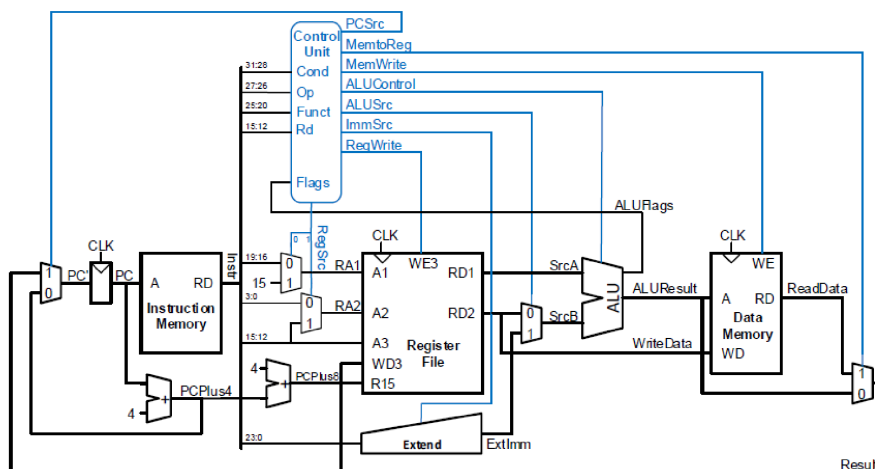
8.1.4 RISC-V 指令集架构的优势

RISC-V 指令集架构的模块化和可扩展性使其在多种应用中具有显著优势：

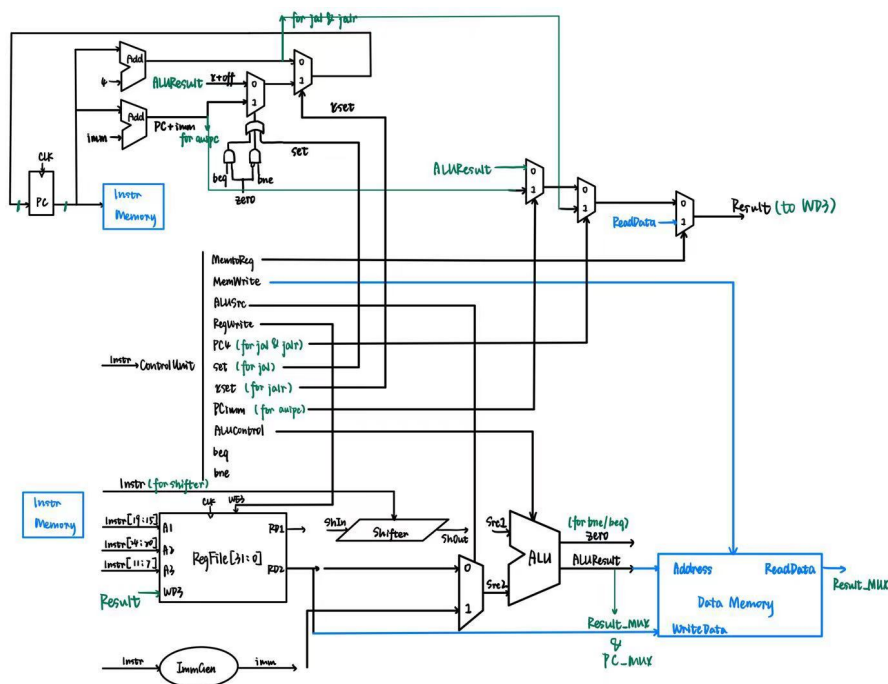
- 简洁性：基础指令集简洁，易于实现和优化，适合资源受限的嵌入式系统。
- 灵活性：通过扩展指令集，可以满足不同应用的需求，从简单的嵌入式设备到高性能计算。
- 开源性：遵循 BSD 协议，允许设计者在不受任何专利限制的情况下自由创新，降低了开发成本。

8.2 RISC-V VS ARM

先前我们从抽象概念的方面上对比了 RISC-V 与 ARM 架构的差别，本学期我们还学习了 ARM 架构的具体搭建，接下来我们从两种架构图的角度具体分析二者之间的差别：



Picture 8.1: 单周期 ARM 架构



Picture 8.2: 单周期 RISC-V 架构

8.2.1 Program Counter

PC 模块中，ARM 指令集中只有 Branch 指令会改变 PC，而 RISC-V 指令集中除了 B 型指令还有 J 型指令改变 PC，因此在 RISC-V 架构中我们单独为 PC 设计了选择器（图 8.2 左上角）以更好地处理不同指令对于 PC 的要求。（如 $PC += 4$ ； $PC += imm$ ； $PC = rs1 + imm$ ）

8.2.2 Control Unit

原 ARM 架构中 Control Unit 需要分出下分为两个解码器，除了对指令进行解码，还需要检测条件与 Flags 是否符合，再输出对 ALU 的控制指令。而 RISC-V 架构中由于不存在 Flags 寄存器，因此只需要有一个对指令进行分析的解码器。

这其中，我们保留了 MemtoReg、MemWrite、ALUSrc、RegWrite、ALUControl 这部分通用指令，去掉了 Branch、ImmSrc、ALUOp 等部分不需要的指令，并为 B 型指令和 J 型指令均设置了专门的输出以解决特定的指令跳转需求。

下面是我们为 RISC-V 的 Control Unit 的解码器编写的真值表：

	op	funct3	MemtoReg	MemWrite	ALUSrc	RegWrite	$x=PC+4$	$pc+=set$	$pc=x+offset$	$x=PC+imm(<<12)$
	op	funct3	MemtoReg	MemWrite	ALUSrc	RegWrite	PC4	set	xset	PCimm
addi	0010011	000	0	0	1	1	0	0	0	0
位移指令	0010011	---	0	0	-	1	1	0	0	0
R-type	0110011	---	0	0	0	1	0	0	0	0
lw	0000011	010	1	0	1	1	0	0	0	0
sw	0100011	010	X	1	1	0	0	0	0	0
beq/bne	1100011	000/001	0	0	0	0	0	0(1)	0	0
(括号内为实际情况，但有前提条件，需要单独讨论)										
jal	1101111	001	0	0	0	1	1	1	0	0
jalr	1100111	001	0	0	1	1	1	0	1	0
auipc	0010111	001	0	0	0	1	0	0	0	1

Picture 8.3: Control Unit 的解码器真值表

8.2.3 Register File

原 ARM 架构中我们的数据读入因为不同指令的存放位置不同，所以我们有前置的选择器进行相关处理。然而 RISC-V 中所有指令的三个寄存器地址都在同一个位置，因此不需要选择器直接读取即可。

此外，ARM 架构中的寄存器仅有 16 个（4 位），然而 RISC-V 架构中存在 32 个（5 位）并且首位（5' b00000）要求始终为 0。

8.2.4 Immediate Data

对于立即数，ARM 指令下同一类型的指令的立即数处理方法一样，因此我们只需要从 Control Unit 确认指令的类型便可分配以相应的处理方式。然而 RISC-V 指令即使是同一类型，立即数的位置也不一样且位置复杂，对此我们的处理方式是直接从代码确认具体的指令，再分配以相应的处理方式。此外，RISC-V 的指令需要额外考虑符号的问题，对于每一个指令的处理还要针对每个 offset 的首位进行分类讨论。

例如下方是对 lw 的指令正负处理代码：

```

1 7'b0000011: begin//LW
2     if(instr[31] == 1) imm = {20'hffff,instr[31:20]};
3     else imm = {20'h0,instr[31:20]};
4 end

```

8.2.5 Shifter

原 ARM 架构中 Shifter 位于 RD2 之后，这是因为相关的指令编排要求是在 A2 读取的数据上进行位移。而在 RISC-V 架构中由于位移指令都是对 A1 读取的数据的处理，因此我们的 Shifter 需要调整位置到 RD1 之后。

同时由于 RISC-V 的寄存器有 5 位，因此我们每一种算法需要多引入一位去参与位移。

8.2.6 ALU

在这里我们没有太多的改动，但是由于 RISC-V 的 B 型指令有要求比较 rs1 与 rs2 是否相等，因此我们在 ALU 同时进行减法运算判定是否为 0 来说明这一点（输出 zero），并将该量输入回 PC 的选择器中与 B 型指令的控制指令联合判断。此外，J 型指令有要求 $PC += imm$ ，我们将这部分借用 ALU 进行计算，直接引出 ALUResult 回到 PC 选择器。

8.2.7 Result MUX

相比于 ARM 架构的 Result MUX，RISC-V 在这方面更加复杂。由于 J 型指令对于我们的写入加入了新的写法（如 $PC+4$ 、 $PC+imm\ll 12$ ），一共四种写法。我们通过 if else 的语法实现写法上的选择，在架构图中表现为 3 个 MUX。

以下为该部分 if else 的代码：

```

1 always @(*) begin
2     if (Branch) begin
3         next_PC = PC_imm;
4     end
5     else if (xset) begin
6         next_PC = ALUResult;
7     end
8     else begin
9         next_PC = PC_plus_4;
10    end
11 end

```

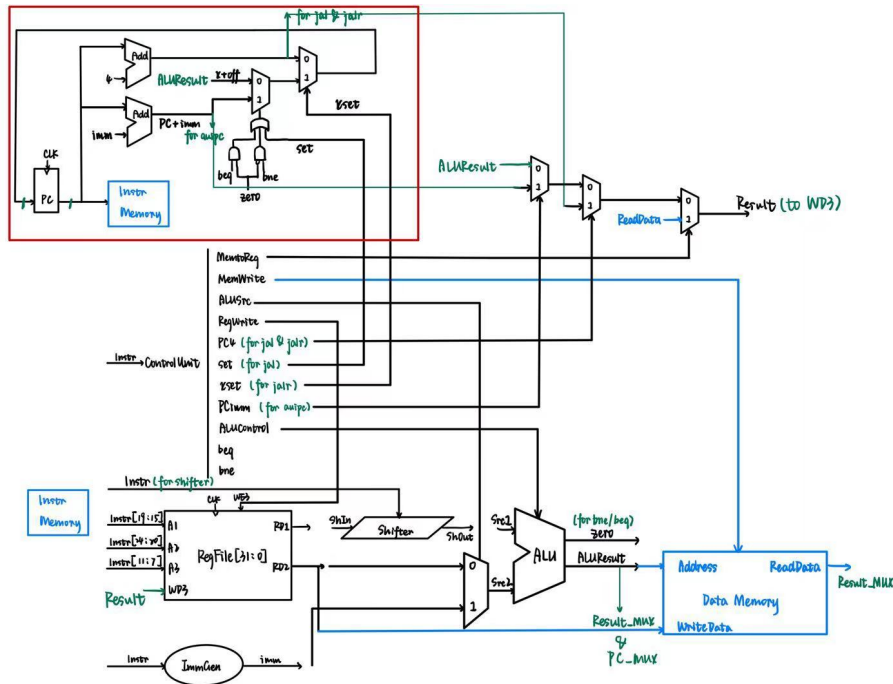
8.3 RISC-V 子模块的详细说明

比较完之后，我们来具体根据 RISC-V 架构来说明各部分的功能及实现方式。

8.3.1 Program Counter

在具体的代码编写中我们将这部分分为 PC 的更新与 PC 的 MUX 两部分。

更新的方式我们依然采用的是异步更新。而 MUX 我们根据对代码的分析有三种写出的可能： $PC+4$ 、 $PC+imm$ 、 $x+offset$ 。其中一个特殊值 $PC+imm$ 为 BNE、BEQ、JAL 三种指令的输出要求，而前两种指令还需要 $rs1$ 与 $rs2$ 相比的结果 (zero)，因此我们设置了一个或门将三种条件相联作为 MUX 的控制信号；另外一个特殊值 $x+offset$ 由 JALR 指令控制，无其他前提条件，因此我们直接从 Control Unit 引入该控制信号至 MUX 中。



Picture 8.4: RISC-V 架构中 PC 部分

8.3.2 Control Unit

在前面我们已经在图 8.3 中列出了 Control Unit 的解码器，这是我们的核心部分。我们根据指令中的 op 进行 case 选择便可以实现大部分的解码功能：

```

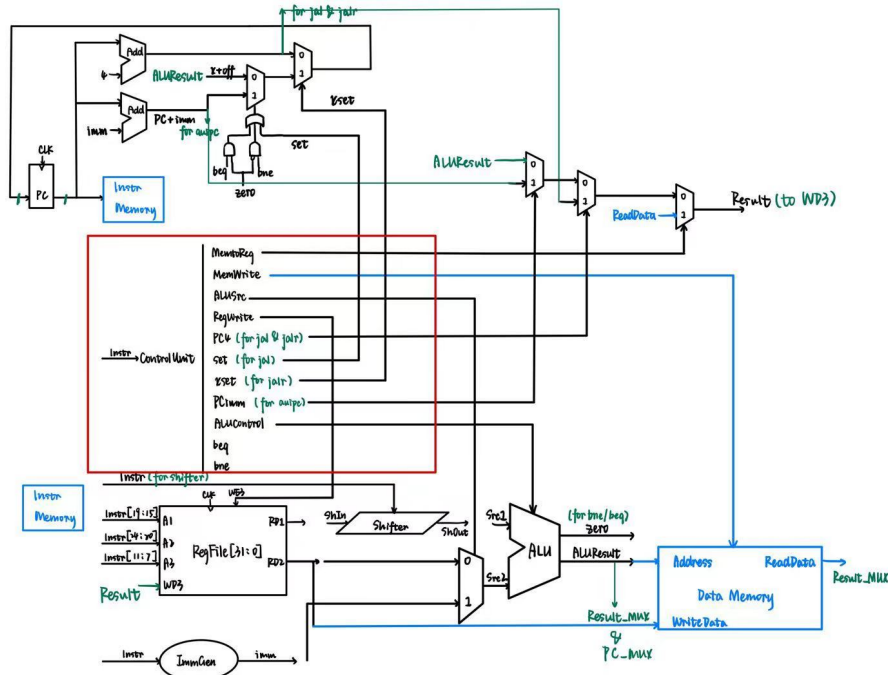
1  always @(*) begin
2      case(op)
3          7'b0010011: {MemtoReg, MemWrite, ALUSrc, RegWrite, PC4, set, xset, PCimm} = 8'b0_0_1_1_0_0_0_0; // addi &
                        位移
4          7'b0110011: {MemtoReg, MemWrite, ALUSrc, RegWrite, PC4, set, xset, PCimm} = 8'b0_0_1_1_0_0_0_0; // R-type
5          7'b0000011: {MemtoReg, MemWrite, ALUSrc, RegWrite, PC4, set, xset, PCimm} = 8'b1_0_1_1_0_0_0_0; // lw
6          7'b0100011: {MemtoReg, MemWrite, ALUSrc, RegWrite, PC4, set, xset, PCimm} = 8'bX_1_1_0_0_0_0_0; // sw
7          7'b1100011: {MemtoReg, MemWrite, ALUSrc, RegWrite, PC4, set, xset, PCimm} = 8'b0_0_0_0_0_0_0_0; // beq/bne
8          7'b1101111: {MemtoReg, MemWrite, ALUSrc, RegWrite, PC4, set, xset, PCimm} = 8'b0_0_0_1_1_1_0_0; // jal
9          7'b1100111: {MemtoReg, MemWrite, ALUSrc, RegWrite, PC4, set, xset, PCimm} = 8'b0_0_1_1_1_0_1_0; // jalr
10         7'b0010111: {MemtoReg, MemWrite, ALUSrc, RegWrite, PC4, set, xset, PCimm} = 8'b0_0_0_1_0_0_0_1; // auipc
11         default: {MemtoReg, MemWrite, ALUSrc, RegWrite, PC4, set, xset, PCimm} = 8'b0_0_0_0_0_0_0_0;

```

```

12     endcase
13 end

```



Picture 8.5: RISC-V 架构中 Control Unit 部分

但是我们还没有产生关于 ALU 的控制信号，这这里我们也需要重新编写相关的真值表：

这里我们可以用 case 选择来实现，但考虑到这里不仅需要去判断 op，还要对 funct3 和 funct7 进行判断，因此代码中我们用 if else 来实现。此外，ALUControl 的功能定义也可以从这张表中得知。

	op	funct3	funct7	ALUOp	ALUControl
addi	0010011	000	-----	1	000
add	0110011	000	0000000	1	000
sub			0100000		001
and		111	0000000		010
or		110			011
xor		100			100
auipc	0010111	001			000(add)
beq/bne	1100011	000/001	-----		001(sub)

Picture 8.6: ALUControl 真值表

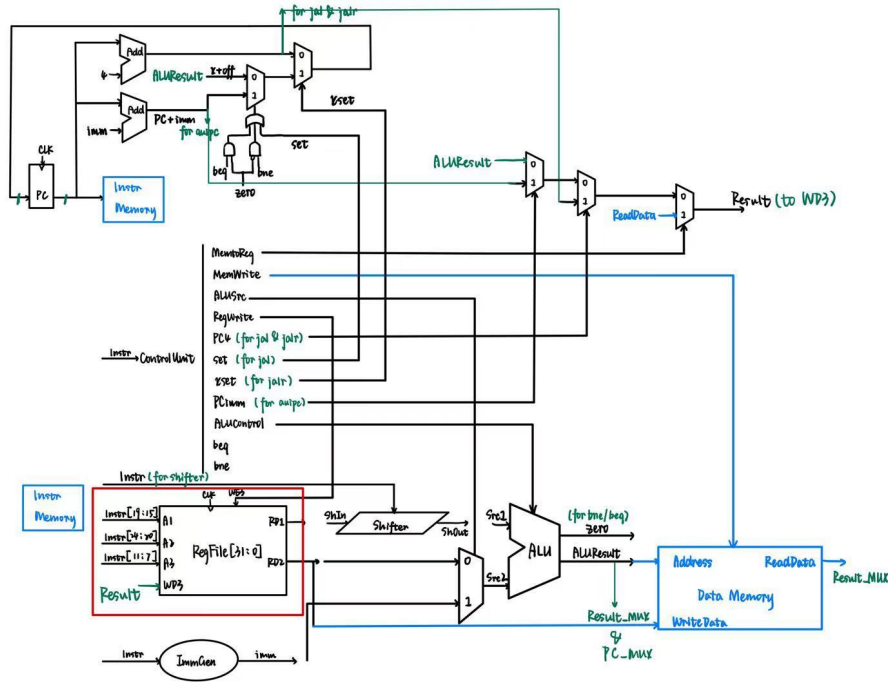
8.3.3 Register File

这部分我们整体采用 ARM 架构中的写法，只是由于要求 Reg[0] 必须一直为 0，因此我们额外对写入这部分加入相关代码：

```

1 if (WE3 & A3 != 5'b0) begin
2     RegBank[A3] <= WD3;
3 end
4 else begin
5     RegBank[0] <= 32'b0; // 永远为0
6 end

```

Picture 8.7: RISC-V 架构中 Register File 部分

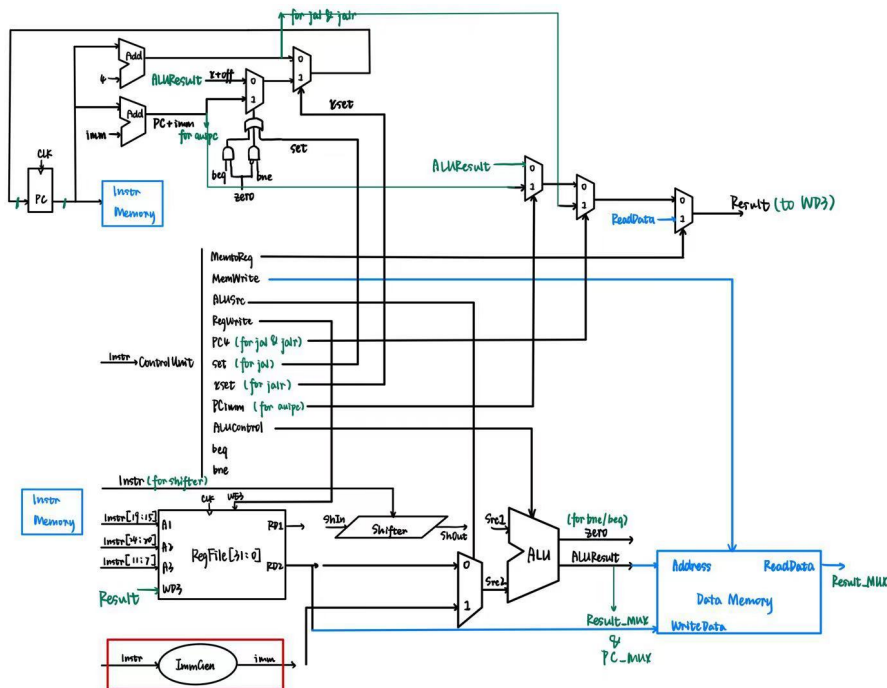
8.3.4 Immediate Data

正如我们先前比较可以得知，我们在这里是直接对代码进行解析而不再依赖 Control Unit 当中的控制信号。根据我们在 Control Unit 的编写经验，我们直接用 case 根据 op 归类出每种指令以及相应的解析。此外，由于每条指令都需要判定立即数的正负性并补全位数，在 case 的各类条件之下我们又设置了 if else 分别归类：

```

1 7'b0000011: begin//LW
2     if(instr[31] == 1) imm = {20'hffff,instr[31:20]};
3     else imm = {20'h0,instr[31:20]};
4 end

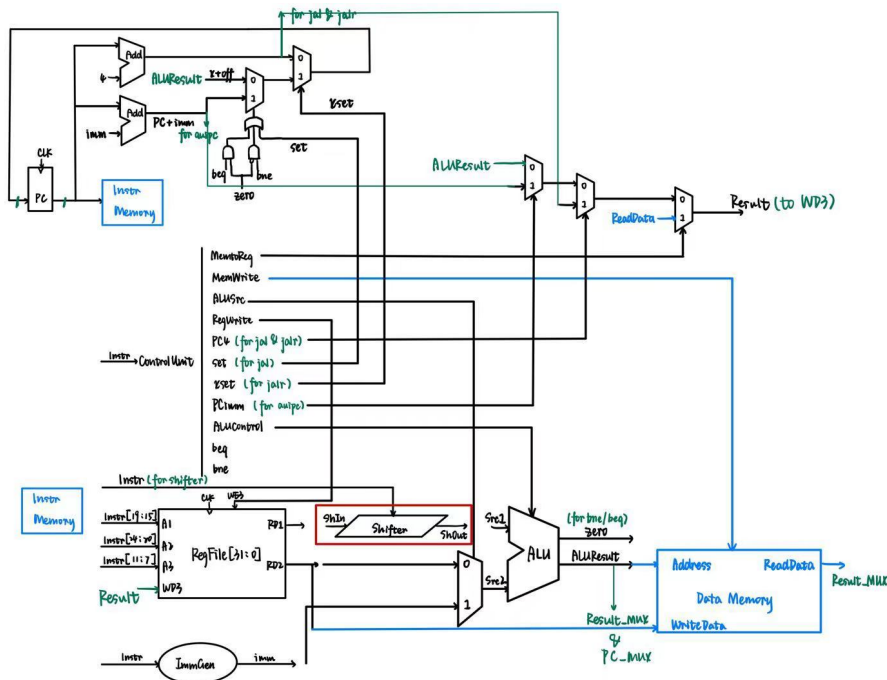
```



Picture 8.8: RISC-V 架构中 Immediate Data 部分

8.3.5 Shifter

我们采用的实现方式同 ARM 架构中的方式一样，采用五重选择器（对应五位）的方式进行位移。考虑到 RISC-V 指令的精简，为了防止错误判断，我们直接引入指令去进行解析判断位移方式。



Picture 8.9: RISC-V 架构中 Shifter 部分

8.3.6 ALU

RISC-V 中我们只有五种运算：加、减、并、或、异或，我们直接用对应符号计算即可。


```

1  3'b000: ALUResult = Src1 + Src2;
2  3'b001: ALUResult = Src1 - Src2;
3  3'b010: ALUResult = Src1 & Src2;
4  3'b011: ALUResult = Src1 | Src2;
5  3'b100: ALUResult = Src1 ^ Src2;

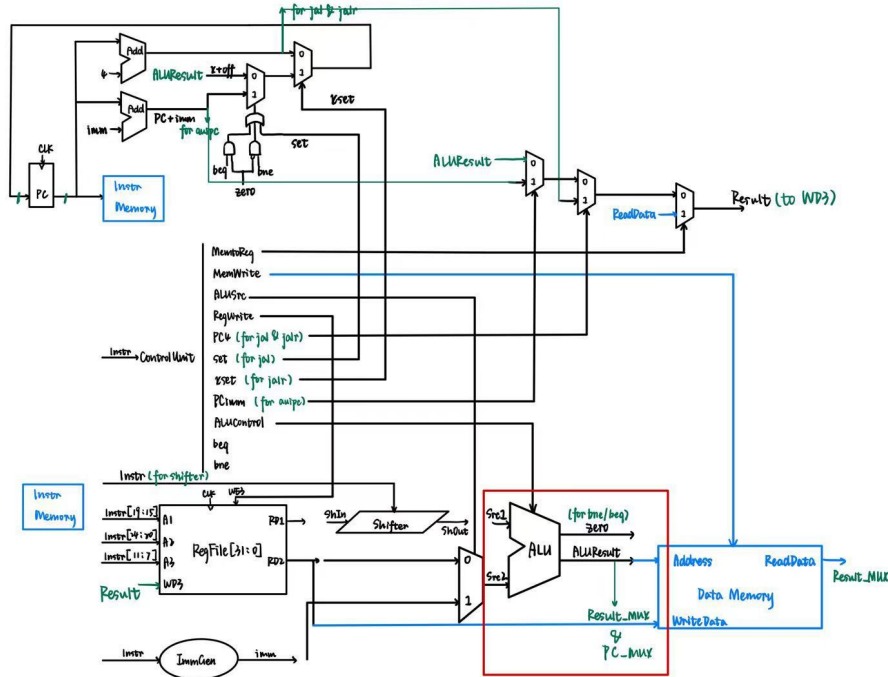
```

根据前面的对比我们引入了一个新的输出量 zero 用于判定是否满足 BNE 或者 BEQ 指令的前提条件:

```

1  assign zero = (ALUResult==32'b0) ? 1'b1: 1'b0;

```



Picture 8.10: RISC-V 架构中 ALU 部分

8.3.7 Result MUX

由于 J 型指令对于我们的写入加入了新的写法 (如 PC+4、PC+imm<12)，一共四种写法。我们通过 if else 的语法实现写法上的选择，在架构图中表现为 3 个 MUX。

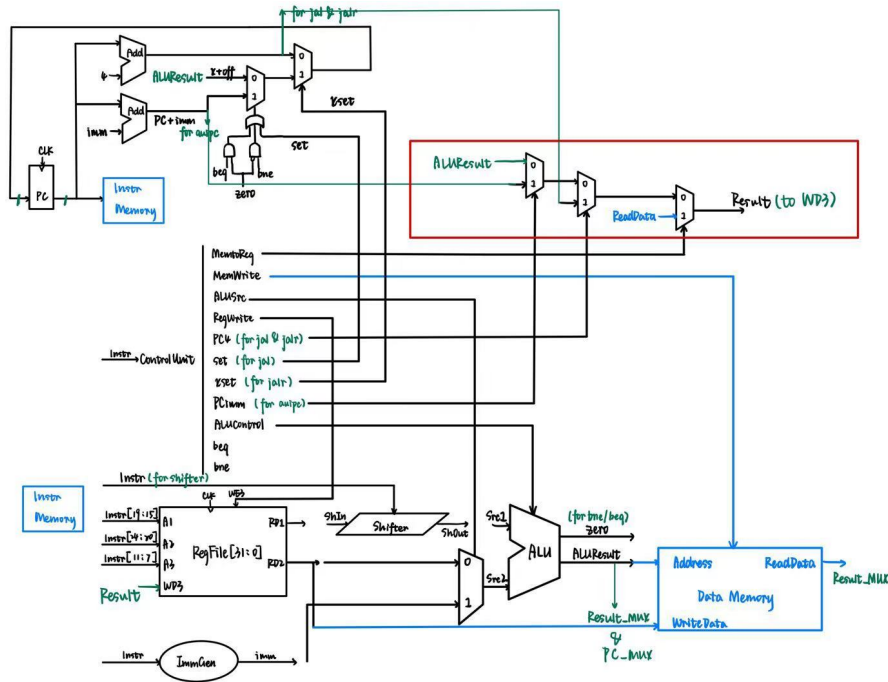
以下为该部分 if else 的代码：

```

1  always @(*) begin
2      if (MemtoReg) begin
3          Result = ReadData;
4      end
5      else if (PC4) begin
6          Result = PC_plus_4;
7      end
8      else if (PCimm) begin
9          Result = PC_imm;
10     end
11     else begin
12         Result = ALUResult;
13     end

```

14 end



Picture 8.11: RISC-V 架构中 Result MUX 部分

8.3.8 Else

我们采用了本学期编写 ARM 架构的 Wrapper 与 Top, 因此关于 Instruction Memory 我们将在后面测试代码中进行展示, 而 Data Memory 模块的编写不做说明。余下仅需进行相关连线将各模块连结即可, 整体相较于 ARM 架构的连接会更加简单。

8.4 测试代码展示和仿真波形分析

以下是我们手动编写的执行指令作为我们测试的代码:

```

1 INSTR_MEM[0] = 32'h0010_0093; //addi x1=x0+1=0+1=1
2 INSTR_MEM[1] = 32'h0020_0113; //addi x2=x0+2=0+2=2
3 INSTR_MEM[2] = 32'h0011_01B3; //add x3=x2+x1=2+1=3
4 INSTR_MEM[3] = 32'h4011_01B3; //sub x3=x2-x1=2-1=1
5 INSTR_MEM[4] = 32'h4020_8233; //sub x4=x1-x2=1-2=-1=补码ffff_ffff
6 INSTR_MEM[5] = 32'h0011_61B3; //OR x3=x2||x1=2||1=3
7 INSTR_MEM[6] = 32'h0011_71B3; //AND x3=x2&x1=0
8 INSTR_MEM[7] = 32'h0011_41B3; //XOR x3=x2^x1=3
9 INSTR_MEM[8] = 32'h0010_9193; //SLLI x3=x1<1=2
10 INSTR_MEM[9] = 32'h0010_D193; //SRLI x3=x1>1=0
11 INSTR_MEM[10] = 32'h4010_D193; //SRAI x3=x1>1=0
12 INSTR_MEM[11] = 32'h4012_5193; //SRAI x3=x4>1=ffff_ffff>1=ffff_ffff
13 INSTR_MEM[12] = 32'h4040_2283; //lw x5=mem[0+1]=mem[1]=820
14 INSTR_MEM[13] = 32'h0020_A323; //sw mem[x1+6]=mem[7]=x2=2
15 INSTR_MEM[14] = 32'h0070_2303; //lw x6=mem[x0+7]=mem[7]=2
16 INSTR_MEM[15] = 32'h0020_8463; //beq x1!=x2

```

```

17 INSTR_MEM[16] = 32'h0032_0463; //beq x3=x4,pc+=8(1000)
18 INSTR_MEM[17] = 32'h0020_8463; //beq x1!=x2
19 INSTR_MEM[18] = 32'h0032_0463; //beq x3=x4,pc+=8(1000)
20 INSTR_MEM[19] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000) NOP
21 INSTR_MEM[20] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000)
22 INSTR_MEM[19] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000) NOP
23 INSTR_MEM[20] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000)
24 INSTR_MEM[21] = 32'h0083_23E7; //jalr pc=x[6]+8,x[7]=pc+4 NOP
25 INSTR_MEM[22] = 32'h0083_23E7; //jalr pc=x[6]+8,x[7]=pc+4
26 INSTR_MEM[23] = 32'h0000_1417; //auipc x[8]=pc+1_0000_0000_0000(1)

```

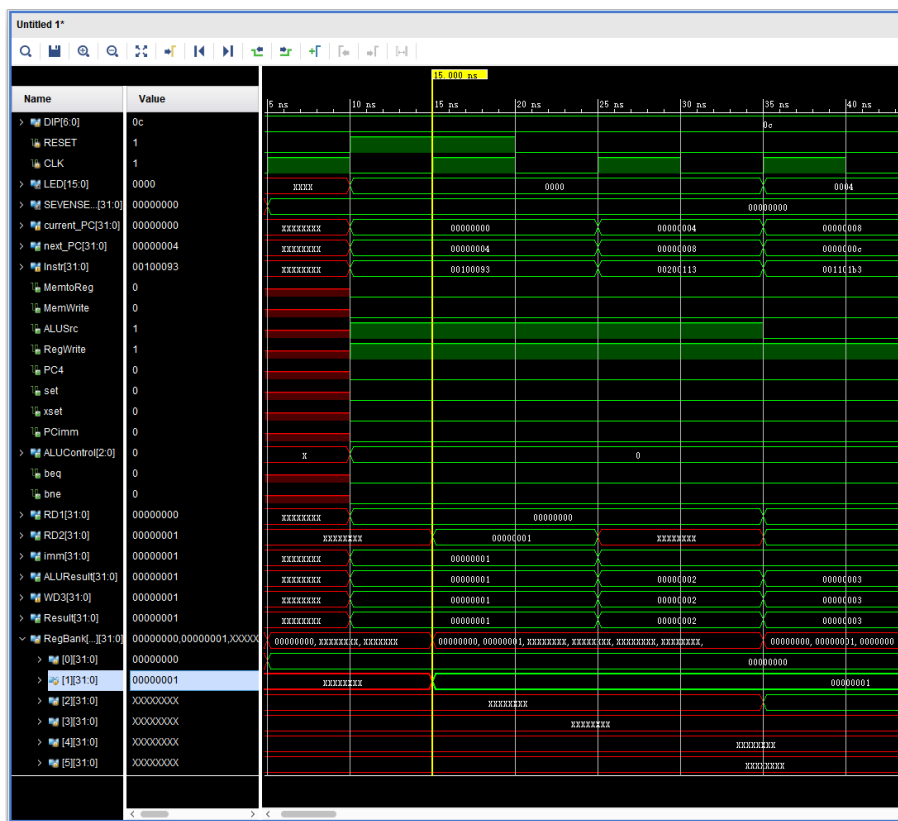
接下来我们将从中挑选出部分指令来解释先前的三类指令的运行方式：

- R 型计算指令：INSTR_MEM[0] = 32'h0010_0093; //addi x1=x0+1=0+1=1

当该指令被读取时，首先会送入 Control Unit 进行解析，向其他部分发送控制信号。同时，Register File 和 Immediate Data 会读取到需要的 rs1 和立即数并完成相应的操作。之后 ALU 会接收到 rs1 和立即数分别作为 Src1 和 Src2。最后计算出来的结果 ALUResult 会写回寄存器 rd 的位置。

对于 PC，将输出 PC+4 的方式从而运行下一条指令。Shifter 将不会运作。

下面是相关的仿真结果：



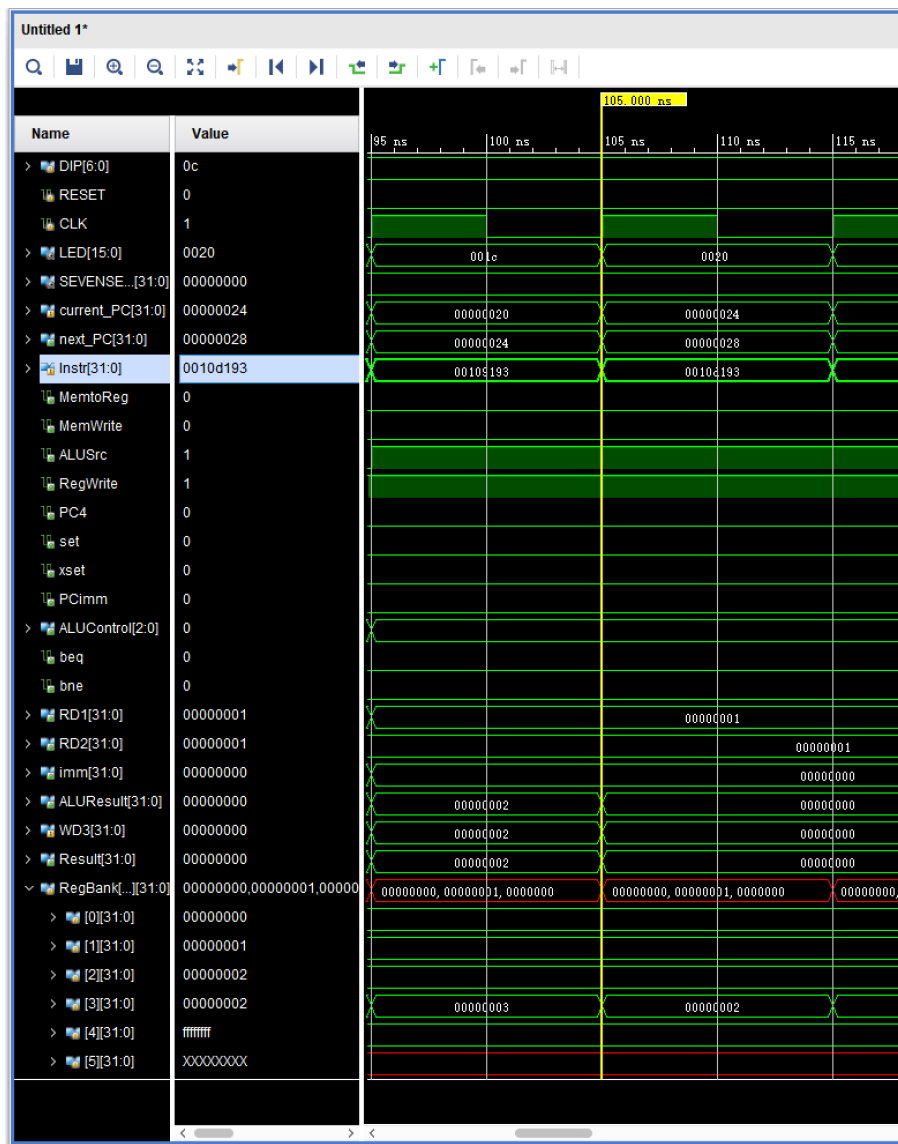
Picture 8.12: R 型计算指令的仿真结果

- R 型位移指令：INSTR_MEM[8] = 32'h0010_9193; //SLLI x3=x1<1=2

当该指令被读取时，首先会送入 Control Unit 进行解析，向其他部分发送控制信号。同时，Register File 会读取到需要的 rs1 并完成相应的操作。之后 Shifter 将会接受到该数据以及该条指令并从中提取出对应的位移数据大小并完成位移要求。之后以位移后的结果 ShOut 输入为 ALU 的 Src1，而 Immediate Data 我们特别地将位移指令的位移数默认为 0，使得它在 ALU 中执行加法的结果能不会改变仅位移的结果。最后 ALU 计算出来的结果 ALUResult 会写回寄存器 rd 的位置。

对于 PC，将输出 PC+4 的方式从而运行下一条指令。

下面是相关的仿真结果：



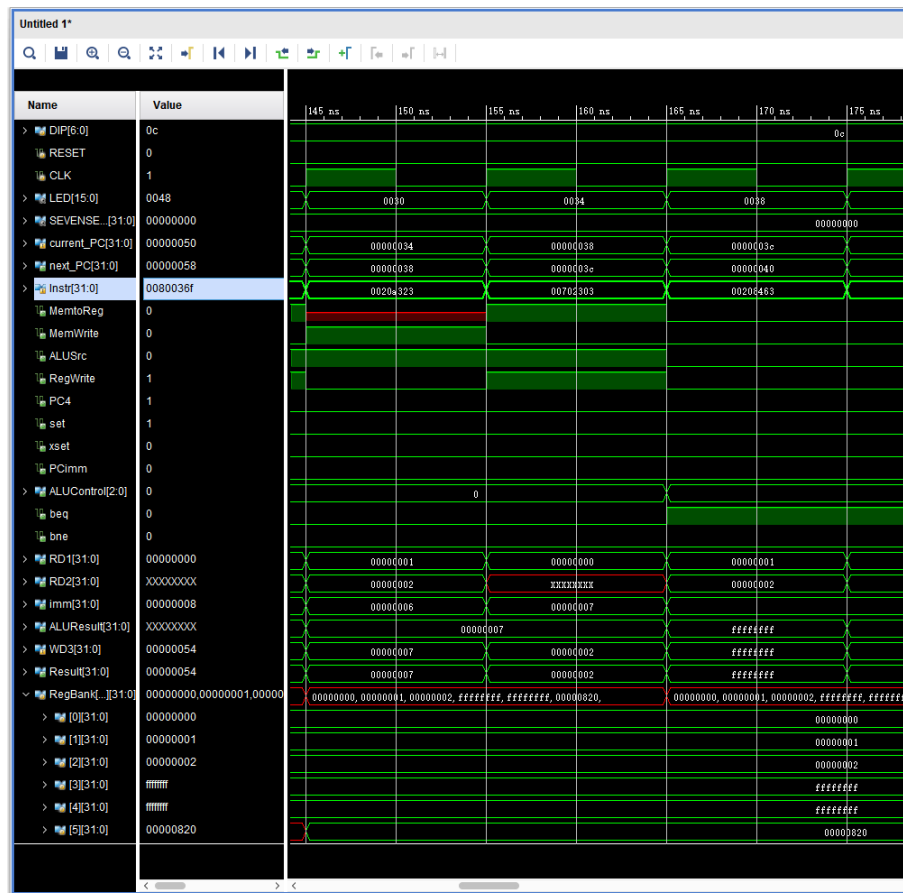
Picture 8.13: R 型位移指令的仿真结果

- L 型加载指令: `INSTR_MEM[12] = 32'h4040_2283; //lw x5=mem[0+1]=mem[1]=820`

当该指令被读取时，首先会送入 Control Unit 进行解析，向其他部分发送控制信号。同时，Register File 和 Immediate Data 会读取到需要的 rs1 和立即数并完成相应的操作。之后 ALU 会接收到 rs1 和立即数分别作为 Src1 和 Src2。最后计算出来的结果 ALUResult 会送入 Data Memory 中进行对应地址的读取。读取出来的数据将作为最终的 Result 写回到 rd。

对于 PC，将输出 PC+4 的方式从而运行下一条指令。Shifter 将不会运作。

下面是相关的仿真结果：



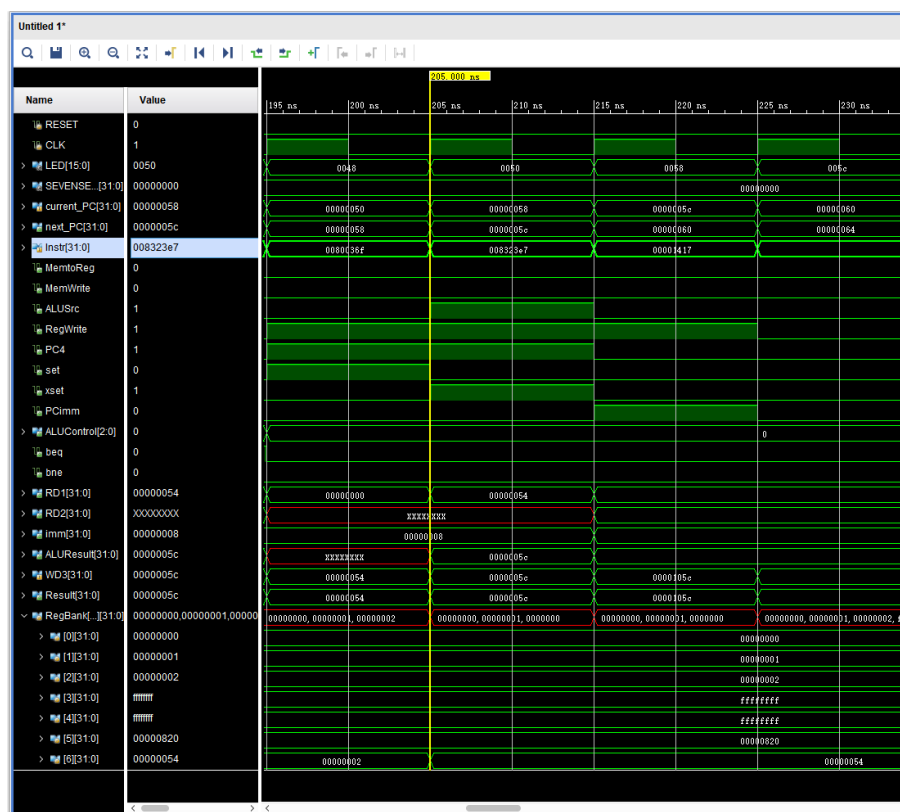
Picture 8.15: S 型写入指令的仿真结果

- B 型指令: `INSTR_MEM[16] = 32'h0032_0463; //beq x3=x4,pc+=8(1000)`

当该指令被读取时, 首先会送入 Control Unit 进行解析, 向其他部分发送控制信号。同时, Register File 会读取到需要的 rs1 和 rs2 并完成相应的操作。之后 ALU 会接收到 rs1 和 rs2 分别作为 Src1 和 Src2。我们将进行减法从而比较这两个数值, 判断是否为 0, 即是否相等。如果这两个数值相等, 则 ALU 输出的 zero 有效。

对于 PC, 由于接收到 BEQ 和 zero 的有效控制信号, 同时接收到 Immediate Data 中的立即数信息并与 PC 进行加和处理, 最后将以代码中编写的输出 PC+8 的方式从而跳过下一条指令。Shifter 将不会运作。

下面是相关的仿真结果:



Picture 8.17: J 型指令的仿真结果

8.5 一个简单的编程项目的运行展示

以下是我们手动编写的一个简单项目的执行指令作为我们测试的代码：

```

1  INSTR_MEM[0] = 32'h0010_0093; //addi x1=x0+1=0+1=1
2  INSTR_MEM[1] = 32'h0020_0113; //addi x2=x0+2=0+2=2
3  INSTR_MEM[2] = 32'h0011_01B3; //add x3=x2+x1=2+1=3
4  INSTR_MEM[3] = 32'h4011_01B3; //sub x3=x2-x1=2-1=1
5  INSTR_MEM[4] = 32'h4020_8233; //sub x4=x1-x2=1-2=-1=补码ffff_ffff
6  INSTR_MEM[5] = 32'h0011_61B3; //OR x3=x2|x1=2|1=3
7  INSTR_MEM[6] = 32'h0011_71B3; //AND x3=x2&x1=0
8  INSTR_MEM[7] = 32'h0011_41B3; //XOR x3=x2^x1=3
9  INSTR_MEM[8] = 32'h0010_9193; //SLLI x3=x1<1=2
10 INSTR_MEM[9] = 32'h0010_D193; //SRLI x3=x1>1=0
11 INSTR_MEM[10] = 32'h4010_D193; //SRAI x3=x1>1=0
12 INSTR_MEM[11] = 32'h4012_5193; //SRAI x3=x4>1=ffff_ffff>1=ffff_ffff
13 INSTR_MEM[12] = 32'h4040_2283; //lw x5=mem[0+1]=mem[1]=820
14 INSTR_MEM[13] = 32'h0020_A323; //sw mem[x1+6]=mem[7]=x2=2
15 INSTR_MEM[14] = 32'h0070_2303; //lw x6=mem[x0+7]=mem[7]=2
16 INSTR_MEM[15] = 32'h0020_8463; //beq x1!=x2
17 INSTR_MEM[16] = 32'h0032_0463; //beq x3=x4,pc+=8(1000)
18 INSTR_MEM[17] = 32'h0020_8463; //beq x1!=x2
19 INSTR_MEM[18] = 32'h0032_0463; //beq x3=x4,pc+=8(1000)
20 INSTR_MEM[19] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000) NOP
21 INSTR_MEM[20] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000)

```

```

22 INSTR_MEM[19] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000) NOP
23 INSTR_MEM[20] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000)
24 INSTR_MEM[21] = 32'h0083_23E7; //jalr pc=x[6]+8,x[7]=pc+4 NOP
25 INSTR_MEM[22] = 32'h0083_23E7; //jalr pc=x[6]+8,x[7]=pc+4
26 INSTR_MEM[23] = 32'h0000_1417; //auipc x[8]=pc+1_0000_0000_0000(1)

```

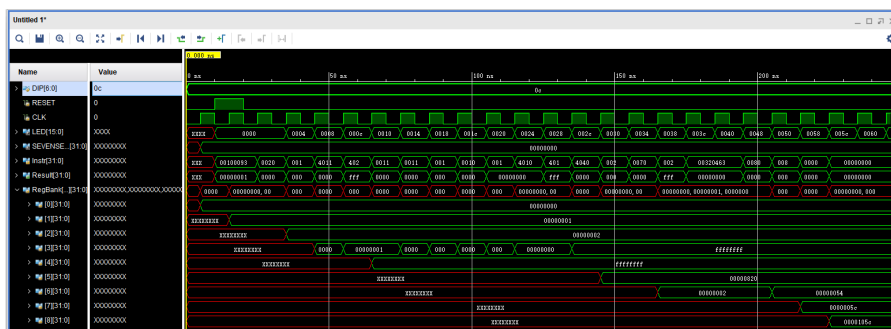
我们沿用了 ARM 架构的测试代码, 但需要说明的是我们没有特别地设计七管显示的数字:

```

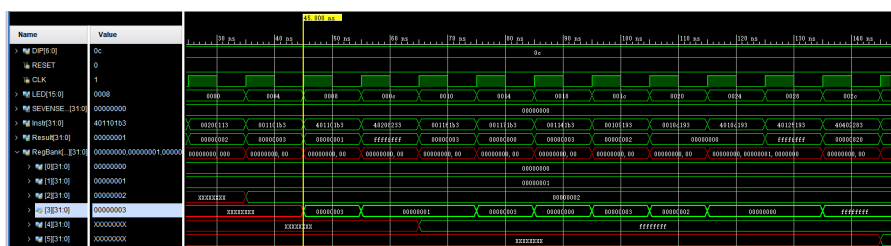
1 module Wrapper_tb;
2 reg [6:0] DIP;
3 reg RESET,CLK;
4 wire [15:0] LED;
5 wire [31:0] SEVENSEGHEX;
6 Wrapper wrapper1 (DIP,LED,SEVENSEGHEX,RESET,CLK);
7 initial begin
8     DIP = 6'b001100;
9     CLK = 0;
10    RESET = 0;
11    #10
12    RESET = 1;
13    #10
14    RESET = 0;
15    #8000
16    $finish;
17 end
18 always #5 CLK = ~CLK;
19 endmodule

```

我们通过代码注释和图中 RegBank 中寄存器、PC 的变化或指令运行顺序的对应我们可以发现本次程序的成功运行。在这里我们将具体展示这个项目运行的结果:



Picture 8.18: 测试结果



Picture 8.19: 测试结果 (局部放大)

8.6 实现过程的困难与收获

8.6.1 遇到的困难

● 真值表的编写兼代码的解析

本次 project 中最复杂的是 Control Unit 中的真值表编写，首先我们需要根据各条指令的最终实现结果进行分类，尽可能减少可能需要的控制指令，将共同的功能进行浓缩融合。通过对指令集的功能分析，我们知悉了部分代码的特殊功能： $x=PC+4$ 、 $pc+=offset$ 、 $pc=x+offset$ 、 $x=PC+imm(\ll 12)$ ，因此我们特别地在 Control Unit 中以它们的功能定义设置了控制信号。此外我们发现不能直接地对 B 型指令设置同 Jal 的形式，因此我们单独再为它们设置了两个信号，当这类代码送入解析后，会激活对应控制信号传输到 PC 的 MUX 中，再结合 zero 对应的判断条件进行检测。

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	$rd = rs1 + rs2$	
sub	SUB	R	0110011	0x0	0x20	$rd = rs1 - rs2$	
xor	XOR	R	0110011	0x4	0x00	$rd = rs1 \wedge rs2$	
or	OR	R	0110011	0x6	0x00	$rd = rs1 rs2$	
and	AND	R	0110011	0x7	0x00	$rd = rs1 \& rs2$	
sll	Shift Left Logical	R	0110011	0x1	0x00	$rd = rs1 \ll rs2$	
srl	Shift Right Logical	R	0110011	0x5	0x00	$rd = rs1 \gg rs2$	
sra	Shift Right Arith*	R	0110011	0x5	0x20	$rd = rs1 \gg rs2$	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	$rd = (rs1 < rs2)?1:0$	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	$rd = (rs1 < rs2)?1:0$	zero-extends
addi	ADD Immediate	I	0010011	0x0		$rd = rs1 + imm$	
xori	XOR Immediate	I	0010011	0x4		$rd = rs1 \wedge imm$	
ori	OR Immediate	I	0010011	0x6		$rd = rs1 imm$	
andi	AND Immediate	I	0010011	0x7		$rd = rs1 \& imm$	
slli	Shift Left Logical Imm	I	0010011	0x1	$imm[5:11]=0x00$	$rd = rs1 \ll imm[0:4]$	
slli	Shift Right Logical Imm	I	0010011	0x5	$imm[5:11]=0x00$	$rd = rs1 \gg imm[0:4]$	
srai	Shift Right Arith Imm	I	0010011	0x5	$imm[5:11]=0x20$	$rd = rs1 \gg imm[0:4]$	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		$rd = (rs1 < imm)?1:0$	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		$rd = (rs1 < imm)?1:0$	zero-extends
lb	Load Byte	I	0000011	0x0		$rd = M[rs1+imm][0:7]$	
lh	Load Half	I	0000011	0x1		$rd = M[rs1+imm][0:15]$	
lw	Load Word	I	0000011	0x2		$rd = M[rs1+imm][0:31]$	
lbu	Load Byte (U)	I	0000011	0x4		$rd = M[rs1+imm][0:7]$	zero-extends
lhu	Load Half (U)	I	0000011	0x5		$rd = M[rs1+imm][0:15]$	zero-extends
sb	Store Byte	S	0100011	0x0		$M[rs1+imm][0:7] = rs2[0:7]$	
sh	Store Half	S	0100011	0x1		$M[rs1+imm][0:15] = rs2[0:15]$	
sw	Store Word	S	0100011	0x2		$M[rs1+imm][0:31] = rs2[0:31]$	
beq	Branch ==	B	1100011	0x0		$\text{if}(rs1 == rs2) PC += imm$	
bne	Branch !=	B	1100011	0x1		$\text{if}(rs1 != rs2) PC += imm$	
blt	Branch <	B	1100011	0x4		$\text{if}(rs1 < rs2) PC += imm$	
bge	Branch $\leq$	B	1100011	0x5		$\text{if}(rs1 \geq rs2) PC += imm$	
bltu	Branch < (U)	B	1100011	0x6		$\text{if}(rs1 < rs2) PC += imm$	zero-extends
bgeu	Branch $\geq$ (U)	B	1100011	0x7		$\text{if}(rs1 \geq rs2) PC += imm$	zero-extends
jal	Jump And Link	J	1101111			$rd = PC+4; PC += imm$	
jalr	Jump And Link Reg	I	1100111	0x0		$rd = PC+4; PC = rs1 + imm$	
lui	Load Upper Imm	U	0110111			$rd = imm \ll 12$	
auipc	Add Upper Imm to PC	U	0010111			$rd = PC + (imm \ll 12)$	

Picture 8.20: RISC-V 指令集

● PC 的 MUX

此外，PC 的 MUX 也比较复杂，为此我们将 PC 的更新和 MUX 单独分为两个部分从而更好地进行逻辑上的分析。先前在真值表的编写中，我们提到 $x=PC+4$ 、 $pc+=offset$ 、 $pc=x+offset$ ，这几个与 PC 相关的变量。因此我们除了要考虑到 PC 自身更新在 MUX 中的功能实现，我们还要考虑引入或者引出一些特殊值满足各类指令在不同模块的需求。因此在这部分我们首先需要准备好各种输入：当前 PC、imm 等变量，同时我们可以借用 ALU 实现其中部分运算，通过逻辑的方式减小硬件上的负担。

● Result 的 MUX

除了 PC 的 MUX，我们在 Result 的 MUX 也涉及了 MUX 的设计。同样的，先前在真值表的编写中，我们提到 $x=PC+4$ 、 $pc=x+offset$ 、 $x=PC+imm(\ll 12)$ ，这几个与 x 相关的变量。因此我们在设计的时候需要思考在哪种合适的位置进行引入或者引出。同样的，我们是否能借用其他模块实现部分运算，减小其他方面的负担。

8.6.2 最终的收获

- **对架构的理解**

相比于从课本上已知的 ARM 架构，这次 RISC-V 架构则是需要 we 根据相关经验自行设计。整个过程并不简单，但我们依然能够从架构的骨相，那些固定的模块出发，从简单功能的实现开始；再在上面添加血肉——Extended Function。整个过程当然不是一帆风顺的，过多的新增变量着实是一番冲击（见困难中对代码的解析），归属哪里，输入输出哪里，当把这些一一排列开来，便有如拨云见月一般的开朗，绘制出最终的架构图像。

- **对代码的理解**

经过这次 project，我们学习了如何自行编写代码解析后的真值表。这个过程无疑是艰难的。但是在逻辑上的抽丝剥茧也是令人难忘的一段记忆。通过细细的分类和多次的修改，最终历数过数次的调整我们实现了现在的真值表，在完成了指定目标的同时，还缩减了不少的代码量，这是非常自豪的。

- **小组之间的互帮互助**

在整个过程中，部分代码初始写法过于复杂，不仅逻辑上太复杂，在代码上也因为行数过多不好进行 debug。幸运的是，我们并不是一个人在做，在逻辑上我们相互交流，进行了有效的精简；在代码上我们相互借鉴，进行了有效的削减。因此最后我们的代码能够足够简单并且排版上也做到足够精致，我们的程序能够顺利完美地运行。